Autonomous Car System

Varun Sreenivasan

ACM Paper: Co-Author

Literature Review - 1

Read the following papers to understand structure of expected paper draft:

1) Reflective Surface for Intelligent Transportation Systems (Li, et. al)
2) Enabling Multiple Applications to Simultaneously Augment Reality: Challenges and Directions (Lebeck, Kohno, Roesner)
3) Augmenting Self-Driving with Remote Control: Challenges and Directions (Banerjee, Kang, Zhao, Qi)
4) Enabling Wideband, Mobile Spectrum through Onboard Heterogenous Computing (Banerjee, Li, Zeng)

Literature Review - 2

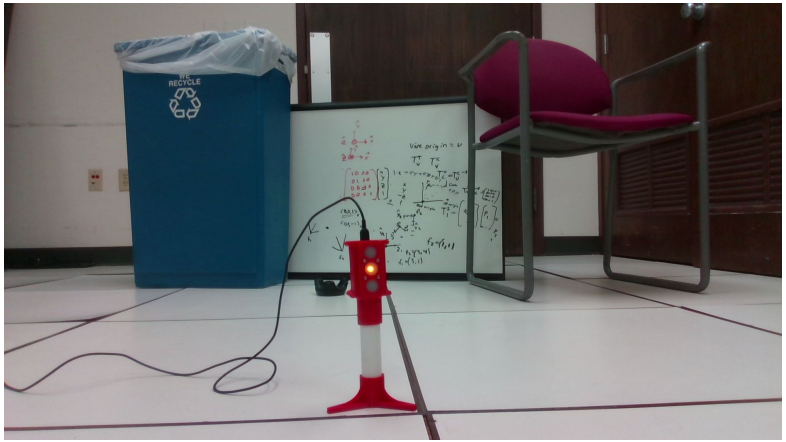
Read the following papers that helped me in writing the paper:

1) You Only Look Once: Unified Real-Time Object Detection (Redmon, et. al)
2) SSD: Single Shot MultiBox Detector (Liu, et. al)
3) MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications (Howard, et. al)
4) Object Detection for Autonomous Vehicles (Lewis)

Dataset

Custom Datasets (generated by us)

- Images for 1 class (traffic light)
- Same method can be used for more general cases
  - Road signs



Annotation Generation

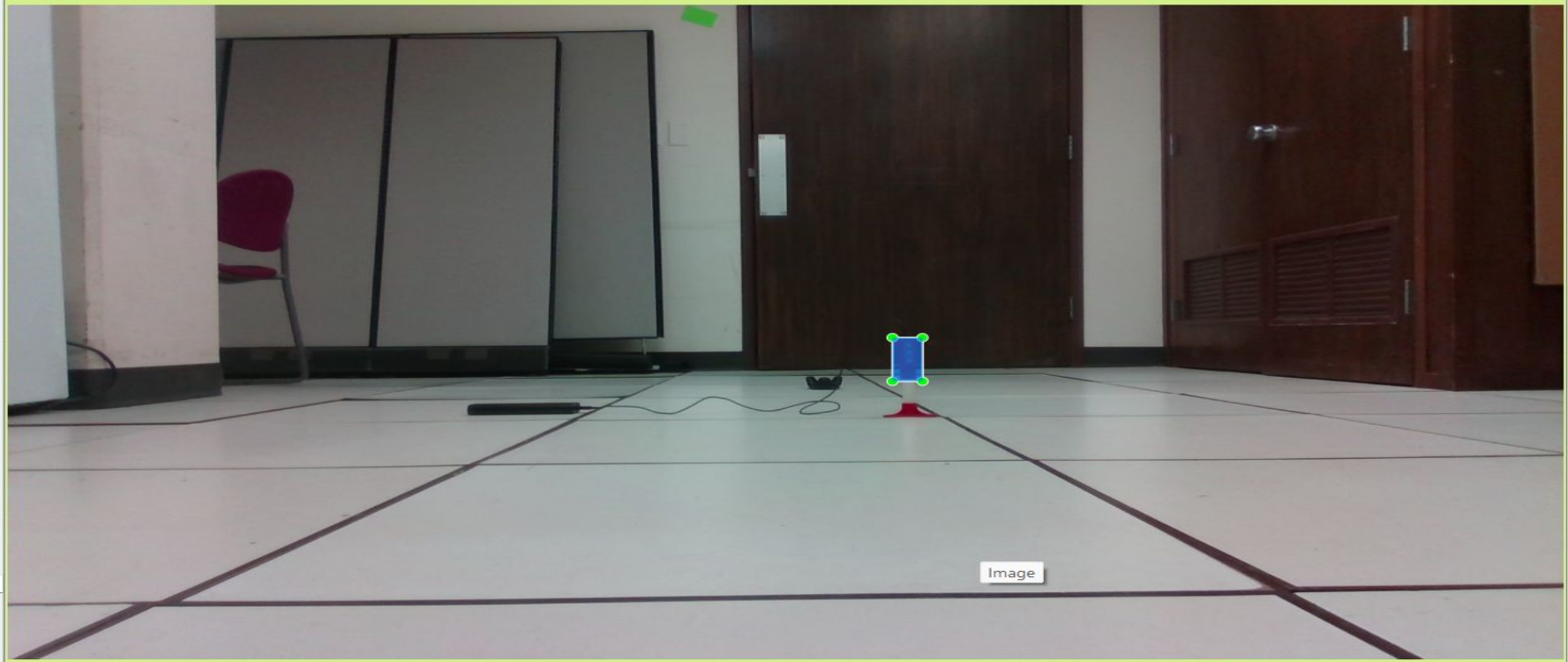
- Identified library to help create annotations for images:
  Tzutalin. LabelImg. Git code (2015). https://github.com/tzutalin/labelImg

- Generated annotations for multiple datasets by using the above library

labelImg C:\Users\vafer\Documents\labelImg\Final-Images\train\color8_0.png [1 / 396]

File Edit View Help

Open
Open Dir
Change Save Dir
Next Image
Prev Image
Verify Image
Save
CreateML
Create RectBox
Duplicate RectBox
Delete RectBox
Zoom In
65 %
Zoom Out
Fit Window
Fit Width



Scripts - Parsing Annotations

A script to parse and consolidate all the individual annotations generated to a single txt file.

Scripts - IOU Computation

A script to compute Intersection Over Union (IOU) for the bounding boxes generated by our algorithm.

Latency Experiment

- In order for a detection system to be considered real-time, it needs to perform inference at a minimum of 30 frames per second.
- I verified that our test framework is indeed real-time by performing a latency test on the SSD-MobileNet model by developing an inference script to compute detection time.
- Our test system is, on average, able to perform detection at a speed of 100 fps, which is comfortably in the real-time range

Writing the paper

Helped in writing, reviewing, and editing the paper.

Object Detection

Model

SSD Mobilenet

- Popular network architecture for real time object detection on mobile and embedded devices.
- Provided by NVIDIA Jetson
- Link: https://github.com/dusty-nv/jetson-inference/blob/master/docs/pytorch-ssd.md

Dataset

Generated another dataset to collect images for multiple backgrounds with stop lights on.

Training

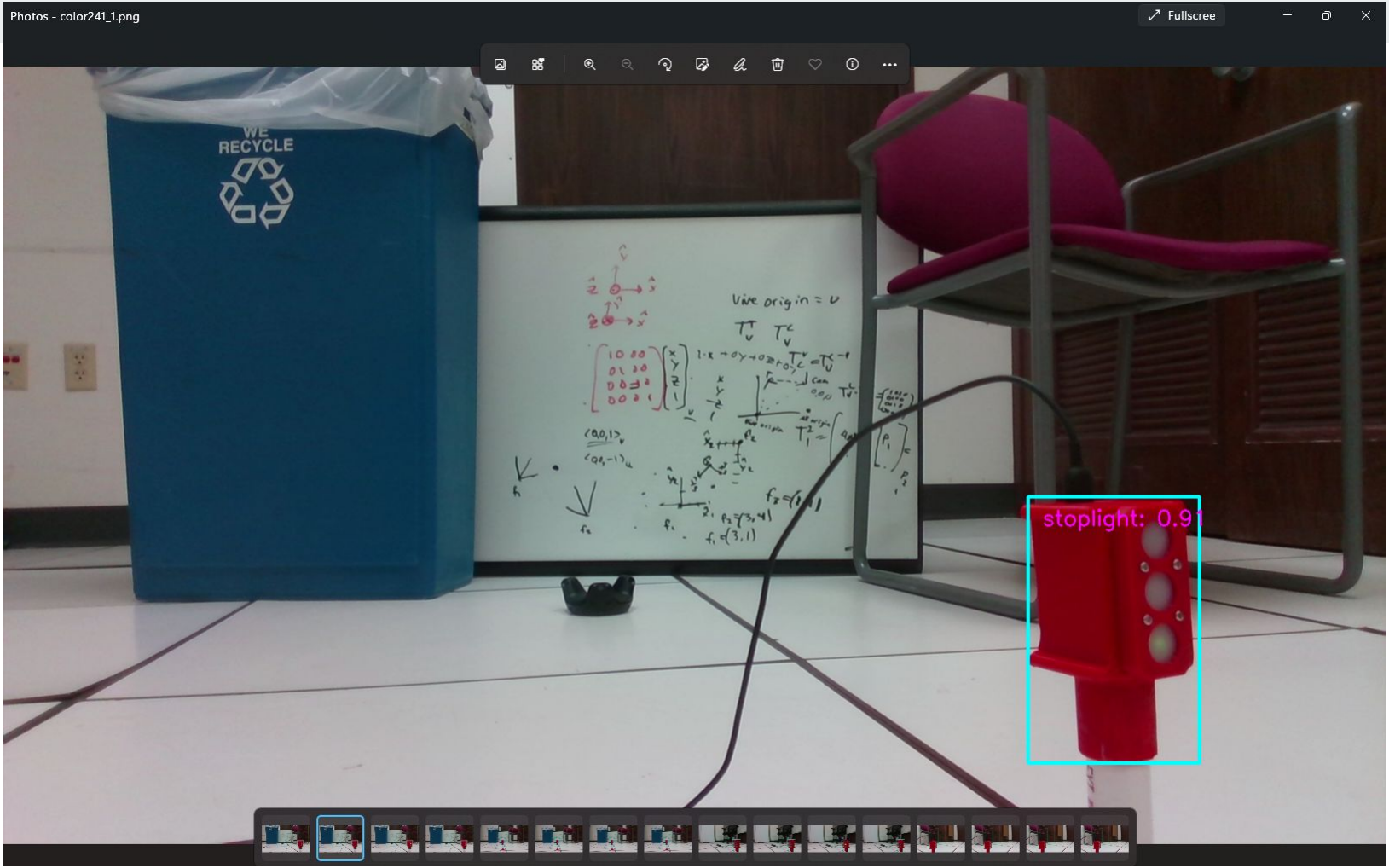
Created inference model using the training set after training for multiple epochs on CSL machines.

Challenges

Creating annotations compliant with Pascal VOC format.

Evaluation on Test Set

Accuracy: 0.33
Recall: 0.18
Precision: 1


Photos - color241_1.png
Fullscree
WE
RECYCLE
stoplight: 0.91

Blob and Color Detection Algorithm

Developed algorithm to detect the traffic light state

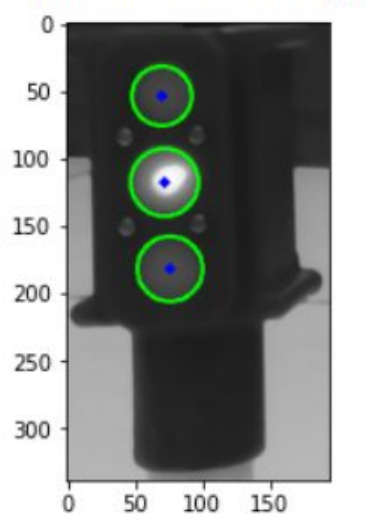
1) Crop image using bounding box generated by SSD-MobileNet detection (annotation passed in as argument to python script along with image name)

2) Run Blob Detection to detect the three circles in the traffic light

3) Run Color Detection to identify state of traffic light

Blob Detection

- Utilized OpenCV's HoughCircles function to detect blobs.
- Fine-tuned hyperparameters on training set images.



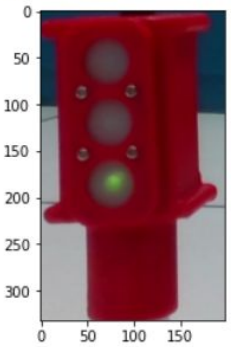
Color Detection

- Utilized OpenCV’s inRange function to detect presence of color.
- Fine-tuned hyperparameters on training set images.

Color Detection - Green

```
annotation = [597, 600, 795, 932 ]
image = 'color237_1.png'
orig_image = cv2.imread(image)[annotation[1]: annotation[3], annotation[0]: annotation[2]]
orig_image = cv2.cvtColor(orig_image, cv2.COLOR_BGR2RGB)
plt.figure()
plt.imshow(orig_image)
main(image, annotation)
```

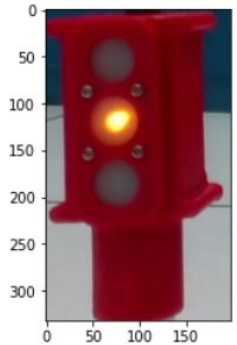
Green



Color Detection - Yellow

```
annotation = [597, 600, 795, 932 ]
image = 'color238_2.png'
orig_image = cv2.imread(image)[annotation[1]: annotation[3], annotation[0]: annotation[2]]
orig_image = cv2.cvtColor(orig_image, cv2.COLOR_BGR2RGB)
plt.figure()
plt.imshow(orig_image)
main(image, annotation)
```

```
Yellow
```



Color Detection - Red

```
annotation = [597, 600, 795, 932 ]
image = 'color239_3.png'
orig_image = cv2.imread(image)[annotation[1]: annotation[3], annotation[0]: annotation[2]]
orig_image = cv2.cvtColor(orig_image, cv2.COLOR_BGR2RGB)
plt.figure()
plt.imshow(orig_image)
main(image, annotation)
```

```
Red
```